



Core Python

Notes



PYTHON

Core Programming Notes

Data Structures • Functions • Modules • Packages

Comprehensive study notes covering Python fundamentals through the scientific computing stack (NumPy, SciPy, Matplotlib, Pandas).

Contents Overview

- 1. Introduction to Python
- 2. Variables, Literals & Data Types
- 3. Built-in Functions & Type Conversion
- 4. Working with Numbers (int, float, complex)
- 5. Strings
- 6. Operators
- 7. Control Flow (conditional, looping, transfer)
- 8. Data Structures – List
- 9. Data Structures – Tuple
- 10. Data Structures – Set
- 11. Data Structures – Dictionary
- 12. Functions
- 13. Modules
- 14. Standard & Mathematical Modules
- 15. Functional Programming Modules (itertools, functools, operator)
- 16. sys & datetime Modules
- 17. Namespaces, Scope & Packages
- 18. Scientific Stack – NumPy, SciPy, Matplotlib, Pandas

1. Introduction to Python

History

- Python was developed by Guido van Rossum at the National Research Institute for Mathematics and Computer Science, Netherlands, in 1990.
- It is derived from many languages such as ABC, Modula-3, C, C++, Algol and SmallTalk, and other scripting languages.
- In December 2008, Python 3.0 (Py3K) was released.

Features of Python

- Easy to learn and use: it is developer-friendly and a high-level programming language.
- Interpreted language: Python provides a shell where we can execute a single command at a time. By being interpreted, the source code is executed line by line and not all at once. Because of this, it is easier to debug code.
- Interactive mode: Python supports GUI applications that can be created and ported to many systems.
- Free and open source: Python is freely available. It can be downloaded from <https://www.python.org/downloads/>. Being open source means the source code is available to the public.
- Portable / Platform independent: Python can run equally on different platforms such as Windows, Linux, Macintosh etc.
- Object-oriented: Python is an object-oriented programming language and supports POP as well as OOP.
- Extensible: if needed, we can write some Python code in other languages like C++. This makes Python an extensible language, meaning it can be extended to other languages.
- Large standard library: there are libraries for regular expressions, documentation generation, unit testing, web browsers, threading, databases, CGI, email, image manipulation, and much more.
- Dynamically typed: the data type of a value is decided at runtime, not in advance. This is why we do not specify the type of a variable while declaring it.

Limitations of Python

- Python is an interpreter-based language, so it is slower than compiler-based languages.
- It is a high-level language, so it uses many layers to communicate with the system.
- Due to the flexibility of data types, Python's memory consumption is high.
- Weak in mobile computing.

- Underdeveloped database access layers.

Applications of Python

- Google's app engine web-development framework uses Python as an application language.
- Maya, a powerful 3D modelling and animation system, provides a Python scripting API.
- Google makes extensive use of Python in its web search systems.
- Popular YouTube video-sharing service is largely written in Python.
- iRobot uses Python in the programming to develop commercial and military robotic devices.
- NSA (National Security Agency) uses Python for cryptography and intelligence analysis.

Uses of Python

- Build a website
- Develop a game
- Computer Vision (face-detection and color-detection) and Image Recognition
- Machine Learning (give a computer the ability to learn)
- Enable Robotics
- Web Scraping (harvest data from websites)
- Data Analysis / Perform Scientific Computing
- Automate a web browser, Perform Scripting

Structure of a Python Program

- Documentation Section (optional) – includes comments, i.e. non-executable statements written with # or `"""..."""`
- Import Section (optional) – used to include built-in or user-defined modules.
- Global Declaration Section (optional) – used to declare global variables / functions.
- Class Section (optional) – data members & member functions; used to define user-defined classes.
- Subprogram Section (optional) – defines user-defined functions.
- Play Ground Section (essential) – the main section of the program where the actual code is written.

Running Python

1. Interactive Mode

- Interactive mode is a command-line shell where each statement is executed like a command.
- It is used for running single lines or blocks of code quickly and conveniently.

- >>> indicates the shell is ready to accept interactive commands.
- Drawback: once we exit the Python shell, the work is lost. It is suitable for learning and testing predefined functionalities, but not for application development.

2. Normal / Script Mode

- Scripted and finished .py files are run in the Python interpreter.
- In the Python shell: File → New File / Ctrl+N to open a blank file to write code.
- Save the script with a .py extension.
- To run the code: Run → Run Module / press F5.

Internal Working of Python

- When the programmer tries to run .py files as instructions, the interactive manner in the Python shell uses the Python Virtual Machine (PVM).
- Flow: Python Code → Syntax Checker → Byte Code → Python Virtual Machine (PVM) → Output. Errors raise a syntax-error or other-error message.
- Interpreter reads a Python expression or statement (source code) and verifies that it is well-formed. As soon as the interpreter sees an error, it halts translation with an error message.
- If the Python expression is well-formed, the interpreter translates it to its equivalent byte code. Byte code is sent to the PVM where the byte code is executed. If any error occurs during execution, it halts with an error message.

2. Variables, Literals & Data Types

Variables

- A variable is a container that stores values.
- It is a way of pointing to a memory location. Variables are used to store values which are saved in memory and can be reused later.
- A variable is given a location in a computer's memory where a value can be saved and changed.
- In Python, we don't need to declare variables explicitly to reserve memory.
- Python is dynamically typed: the type of a variable is assigned when a value is assigned. A variable is created when a value is first assigned to it (using a variable=value statement).

```
x = 10
x = 10.5
x = "Indore"
```

- Multiple Assignment: we can assign values to multiple Python variables in one statement.

```
x = y = z = 5      # same value to many variables  
a, b, c = 3, 5, "Indore"  # different values
```

- Swapping variables: swapping means interchanging values; in Python swapping is very simple.

```
a, b = b, a
```

- Deleting variables: we can delete a Python variable using the keyword "del".

```
del a
```

Literals

- A literal is a fixed or constant value that appears in a program; it is referred to as literal.
- Literals are numbers or strings or characters that appear directly in a program.
- 1. Int: e.g. 5, 1, -7, -19 etc.
- 2. Long: 345621L (Python 2)
- 3. Float: 4.67
- 4. Complex: 3+5j
- 5. Boolean: True or False
- 6. Special Literal None: represents the absence of a value or null value, e.g. "x=None".
- 7. String Literals: string literals can be formed using single, double or triple quotes, e.g. "Hello", 'Hello'.
- 8. Collections: collections such as tuples, lists and dictionary are used in Python.

Collections

- List: contains items of different data types. Lists are mutable; values stored in a list are separated by a comma (,) and enclosed within square brackets [].
- For ex.: [90, 102, 67.5]
- Tuple: a tuple is a sequence of immutable Python objects. Tuples can be created using round brackets () and items are separated by a comma. For ex.: (90, 102, 67.5)
- Dictionary: the dictionary in Python is an unordered set of key:value pairs created using curly brackets { }. Any and value are separated by colon ":" and key:value pairs are separated by a comma.
- For ex.: {'string':'Hello', 'age':7}

- Key and value are separated by colon (:); items are separated by commas; the whole is enclosed in { }.

Data Types in Python 3

- A data type is one that can be stored in an identifier / variable; it is known as its data type.
- A data type determines how much memory is allocated to store data and what operations can be performed on it.
- Although we don't have to declare a type for Python variables, a value does have a type.
- Python supports the following built-in data types.

Type	Used to represent	Mutability	Example
int	Whole numbers (no fractional part)	Immutable	a = 25 → type(a) → <class int>
float	Real numbers / floating-point (with fraction)	Immutable	a = 25.5 → <class float>
complex	Complex numbers (real + imaginary, written as a+bj)	Immutable	a = 3+5j → <class complex>
bool	Logical values True or False	Immutable	flag = True → <class bool>
str	Sequence of characters	Immutable	s = 'Hi' → <class str>
bytes	Sequence of bytes (0–255)	Immutable	a = bytes([10,20,30,40])
bytearray	Like bytes but mutable	Mutable	a = bytearray(...)
range	Sequence of numbers	Immutable	r = range(10) → <class range>
list	Ordered collection of objects	Mutable	a = [10,20,30] → <class list>
tuple	Ordered collection of objects	Immutable	a = (10,20,30) → <class tuple>
set	Unordered collection of unique objects	Mutable	a = {10,20,30} → <class set>
frozenset	Unordered collection of unique objects	Immutable	a = frozenset({10,20})
dict	Collection of key:value pairs	Mutable	a = {101:'Kiran',102:'Ankit'}
NoneType	Represents the absence of a value	Immutable	x = None → <class NoneType>

Immutability – "In Python everything is an object"

- In Python everything is an object. Immutable object means "once an object is created, we cannot modify it".

- If we try to modify an immutable object, PVM will delete the existing object and create a new object.
- In Python, if a new object is required then the PVM doesn't create a new object immediately; first it will check whether a new object already exists in memory and it will be reused with new reference.
- However, problem with the above approach is that several references pointing to the same object in memory will be created. If we change any one reference then remaining references will be affected. To prevent this, immutability is introduced.
- Note: catching is not applicable to float, complex, mutable objects, collection types.

Constants

- In Python, everything is an object.
- A constant concept is not applicable in Python.
- But it is convenient to use only upper-case characters if we don't want to change.
- For example: MAX_VALUE = 10
- It is just a convention but we can change the value.

3. Built-in Functions (BIF) & Type Conversion

BIF (Built-in Functions)

- Python contains around 67 built-in functions to perform basic operations.
- We are going to study them here so we see them in operation.

1. type()

- type() returns the data type of an object (variable).
- Syntax: type(object)

2. id()

- id() returns an object's identity (the address of an object).
- Syntax: id(object)

3. print()

- It is used to print output.
- We can print almost anything by using this powerful BIF.
- Syntax: print(object)

4. input()

- input() reads and returns a line of input.
- It is used to accept input as a string from user.
- We need to apply type conversion to input by any other type.
- Syntax: input() / input("Message for user")
- Reading multiple values in a single statement: a,b = [int(x) for x in input("Enter 2 Numbers").split()]

5. eval()

- eval function takes a String and evaluates the result.
- eval() can evaluate the input to int, float, bool, list, tuple, etc. as appropriate, based on the detected type.
- Example: a,b = [int(x) for x in input("Enter 2 Numbers").split()]

Type Conversions

- Since Python is dynamically typed, we may want to convert a value into another type.
- Python supports a lot of functions for the same.

Function	What it does	Example
int()	Converts the value into an int. String with only digits can be converted.	int('25') → 25
float()	Converts the value into a float. String with only digits can be converted.	float('4') → 4.0
str()	Converts an object into a string.	str(7.1) → '7.1'
bool()	Converts a value into Boolean. Non-Zero value → True, Zero or empty → False.	bool(0) → False
complex()	Converts value into complex. Only iterable objects allowed.	complex(1,2) → 1+2j
list()	Converts a value into a list. Only iterable objects allowed.	list('xyz') → ['x','y','z']
tuple()	Converts a value into a tuple. Only iterable objects allowed.	tuple([1,2,3]) → (1,2,3)

4. Working with Numbers

Playing with Integers (int data type)

- An integer is a whole number or number without any decimal part.
- In Python, an integer can be of any length allowed by available memory space.
- There is no limit to int range. As long as you have enough memory you can store as many digits as composes memory space allows.
- We can represent an integer in the following formats:

Format	Base	Prefix	Function
Decimal	10	N/A (it is default)	—
Binary	2	0b or 0B	bin()
Octal	8	0o or 0O	oct()
Hexa-decimal	16	0x or 0X	hex()

```
>>> x = 250
>>> print(bin(x))
0b11111010
>>> print(oct(x))
0o372
>>> print(hex(x))
0xfa

>>> x = 0b11111010 # binary literal
>>> print(x)
250
```

Playing with Real numbers (float data type)

- A floating-point number is a real number with a decimal point or fractional part.
- A floating-point number is accurate up to 15 decimal places.
- We can also represent a float value in exponential form using the character "e" or "E".
- Example: $x = 1.2e3 \rightarrow 1200.0$
- We can use only decimal form for float values.

Playing with Complex numbers (complex data type)

- Complex numbers are written in the form $a+bj$, where a is the real part and b is the imaginary part.
- If the real part is missing, we can specify it directly as bj .
- However, an imaginary part can only be written in decimal or binary or octal or hexa form.



Advance Python

Notes



PYTHON

Advanced Programming Notes

Data Structures • Functions • Modules • Packages

Comprehensive study notes covering Python fundamentals through the scientific computing stack (OOP, File Handling, Database, Exception Handling, Multithreading & GUI).

Python (Advance) — Notes

Topics: OOP, File Handling, Database, Exception Handling, GUI

1. Object-Oriented Programming (OOP)

- In OOP, emphasis is on **data** rather than procedure.
- OOP treats data as the critical element in program development.
- OOP does not allow data to flow freely around the system. It binds data more closely to the functions that operate on data.
- In OOP, a program is divided into a number of smaller entities called **objects**.
- Each object contains data and functions to manipulate its data.
- Data of an object can be accessed only by functions associated with that object. However, multiple objects may communicate with each other through functions. This is known as **message passing**.

Object

- Object is a **basic run-time entity** in an object-oriented system.
- An object may represent a book, a person, a place, a bank account, or any important data item that a program has to handle.
- Any object can be represented by its data (attributes) and behaviors (operations).
- To create objects, we require some model or plan or blue print, which is nothing but a **class**.
- Data properties can be represented by **variables**.
- Actions or behaviors can be represented by **methods**.
- Hence, a class contains both variables and methods.

Class

- An entire set of data and functions can be made a user-defined data type with the help of a class.
- A class is like a blue-print of an object.
- *"Class is a user-defined data type which behaves like a fundamental data type."*
- An object is an instance of a class.
- Once a class is defined, we can create any number of objects.

Encapsulation

- *"The wrapping up of data and functions into a single unit (i.e. class) is referred as encapsulation."*
- Encapsulation is one of the important features of class.
- Data encapsulated in a class is accessible only to the functions inside the class, such insulation of data from direct access by outside functions is referred as **Data Hiding**.

Abstraction

- *"Abstraction refers to the act of representing essential information without including background details."*
- Classes use the concept of abstraction and are declared as an abstract list of attributes.

Inheritance

- *"The mechanism of creating a new class from an existing class is referred as inheritance."* or
- *"When one class acquires the properties of another class, it is an inheritance."*
- The existing class is referred as **base class** or **parent class** or **super class**.
- The new class is referred as **derived class** or **child class** or **sub class**.
- In OOP, inheritance provides the idea of **reusability of code**.

Polymorphism

- *"The ability to take multiple forms with a single name is referred as polymorphism."*
- Polymorphism plays an important role in allowing objects having different internal structures to share the same external interface.

2. Syntax to Create a Class

```
class ClassName:  
    "documentation string"  
    variables  
    methods
```

- **class** is a keyword.
- **ClassName** is a type name given by the programmer.
- The documentation string describes the class. It is optional.
- BIF **help(ClassName)** on the class uses this documentation string.

- Within a class we can represent data by using **variables**.
- There are 3 types of variables:
 1. Instance Variables (Object Level Variables)
 2. Static Variables (Class Level Variables)
 3. Local Variables (Method Level Variables)
- Within a class we can represent operations by using **methods** (functions).
- There are 3 types of methods:
 4. Instance Methods
 5. Class Methods
 6. Static Methods

Object

- An object is the physical existence of a class.
- An object is an instance of a class.
- **Syntax:**

```
objName = ClassName()
```

- The identifier with which an object is referred is called a **reference variable**.
- By using reference variable (objName), we can access members (attributes) of an object.
- Get the operator to access members of object: `objName.method()`
- We can create any number of objects from a class.

Parameter self (self-variable)

- **self** is a parameter which always points to the current object (like the **this** keyword in C++/Java).
- The first parameter inside a constructor or any instance method of a class is treated as **self**.
- By using **self**, we can access members of the object within the class.

```
class Test:
    "This is test class"          # Documentation string
    def method(self):             # Instance method
        print("[object] called me")

>>> a = Test()                  # Object creation i.e. instantiation
>>> b = Test()
```

```
>>> a.method()          # method call
>>> b.method()          # method call
```

- Observe output carefully to understand the self-variable.
- When **a** calls method, **self** is reference for object **a**.
- When **b** calls method, **self** is reference for object **b**.

3. Constructor

- A constructor is a special method in a Python class.
- The name of the constructor should be `__init__`.
- The constructor will be executed automatically at the time of object creation.
- The main purpose of a constructor is to declare and initialize instance variables.
- The constructor will be executed once per object.
- A constructor takes at least one argument (i.e. **self**).
- A constructor is optional. If we don't provide any constructor, then it is added automatically.

```
class Test:
    "This is test class"          # Documentation string
    def __init__(self):          # Constructor
        print("[object] is created")

>>> a = Test()                  # Creating object
>>> b = Test()
```

Array of objects / List of objects

- If we want to create a collection of multiple objects, then we can create a list of objects.

```
>>> a = [Test(), Test(), Test()] # objects created, printed through
constructor
```

- Alternatively, we can create an empty list and append objects to it.

```
>>> a = []
>>> for i in range(3):
...     t = Test()
```

```
...     a.append(t)
```

4. Destructor

- Destructors are called when an object is destroyed.
- In Python, destructors are not required as much as they are required in C++, because Python has a **garbage collector** that handles memory management automatically.
- The `__del__()` method is known as the destructor method in Python.
- It is called when all references to the object have been deleted, i.e. when an object is garbage collected.
- `getrefcount` function of the `sys` module can be used to check reference count.
- Just before destroying an object, the Garbage Collector calls the destructor to perform cleanup activities (like closing database connection).
- Once the destructor is executed, then the Garbage Collector automatically destroys that object.
- If we don't write a destructor, then **it is not added automatically**.

```
class Sample:
    def __init__(self):
        print("Constructor Execution...")
    def __del__(self):
        print("Destructor Execution...")

s = Sample()
del s
```

5. Garbage Collector

- In old languages like C++, the programmer is responsible for both creation and destruction of objects (dynamic).
- Programmers usually forget to delete useless objects, which leads to out-of-memory errors.
- In Python, we have an assistant which is always running in the background to destroy useless objects.

- Because of this assistant, the chance of a Python program failing with memory problems is very less. This assistant is nothing but the **Garbage Collector**.
- The Garbage Collector is enabled by default, but we can disable it based on our requirement.
- We can use the following functions of the `gc` module:

```
import gc
gc.isenabled()    # True if gc is enabled
gc.disable()      # disables gc
gc.enable()       # enables gc
```

6. Types of Variables in a Class

There are 3 types of class variables:

1. **Instance Variables** (Object Level Variables)
 2. **Static Variables** (Class Level Variables)
 3. **Local Variables** (Method Level Variables)
- **Data member** — a class variable or instance variable that holds data associated with a class and its objects.

Instance Variable

- Instance variables are defined in the constructor or any instance method of a class.
- For every object, a separate copy of instance variables will be created.
- The value of an instance variable is varied from object to object (object specific).
- Any changes made to an instance variable of one object **don't reflect** in other objects of the class.
- We can access instance variables within the class by using the self-variable and outside of the class by using the object reference.

How & where to create Instance Variables?

- **Inside the constructor** by using the self-variable. Once we create the object, automatically these variables will be added to the object.
- **Inside an instance method** by using the self-variable. Instance variable will be added when we call that method.

- **Outside of the class** by using the object reference.

```
class Test:
    def __init__(self):        # constructor
        self.a = 10          # instance variable
    def method(self):         # instance method
        self.b = 20          # instance variable

>>> t = Test()               # object creation
>>> t.__dict__
{'a': 10}
>>> t.method()
>>> t.__dict__
{'a': 10, 'b': 20}
>>> t.c = 30                 # outside the class
>>> t.__dict__
{'a': 10, 'b': 20, 'c': 30}
```

Deleting an instance variable

- **Inside a class** (in an instance method):

```
del self.variableName
```

- **Outside of the class:**

```
del object.variableName
```

- Note: The instance variables which are deleted from one object will not be deleted from other objects.

Static Variable / Class Variable

- If any variable has a common value for all objects of a class, then such a variable should be declared as a static or class variable.
- Variables defined inside a class but outside of instance methods are static variables or class variables.
- These are class level variables. All objects of a class have a common value for static variables.
- For a class, only one copy of a static variable will be created and shared by all the objects of that class.

- We can **access** static variables either by class name or object reference. But generally, we use the class name.
- By using object/self we can read static variables, but we cannot modify or delete them.
- We can modify or delete static variables only by using the class name or the `cls` variable in a class method.

How & where to create static Variables?

- Inside the body of the class directly.
- Inside the constructor by using the class name.
- Inside an instance method by using the class name.
- Inside a class method by using either the class name or the `cls` variable.
- Inside a static method by using the class name.

Deleting a static variable

- **Anywhere in the program:**

```
del ClassName.variableName
```

- **Inside a class method:**

```
del cls.variableName
```

```
class Test:
    myclass = 35          # static variable

>>> Test.myclass         # access through class name
35
>>> t1 = Test()
>>> t1.myclass           # access through object
35
>>> t2.myclass           # access through object
35
```

Local Variable

- Sometimes, to meet temporary requirements of a method, we can create variables inside a method directly. Such variables are called **local variables** or temporary variables.
- Local variables will be created at the time of method execution and destroyed once the method completes.

- Local variables of a method cannot be accessed from outside of the method.
- These are not data members of an object or class.

7. Types of Methods in a Class

There are 3 types of methods:

4. **Instance Methods**
5. **Class Methods**
6. **Static Methods**

- **Method** — a function defined in a class to manipulate the data of an object or class. Also known as a member function.

Instance Methods

- These are most commonly used methods in a class for managing data attributes of an object.
- It is a function defined normally inside a class **without any decorator**.
- We can call instance methods by using object reference from outside.
- Object reference is passed as the first parameter (self) to instance methods.
- We can access instance variables inside an instance method by using self (first parameter) **self**.
- Within a class, an instance method can call another instance method by using self.

Program (Instance method example)

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks
    def display(self):
        print("Hi", self.name)
        print("Your marks are:", self.marks)
    def grade(self):
        if self.marks >= 60:
            print("You got First Grade")
        elif self.marks >= 50:
            print("You got Second Grade")
        elif self.marks >= 35:
```

Download Now
Full PDF
Only Core Python
₹29

BUY

<https://rzp.io/rzp/1-note-1>

[#techbywebcoder](#)

Purchase Now
Full PDF
Core + Advance Python
₹49

BUY

[**https://rzp.io/rzp/2-note-1**](https://rzp.io/rzp/2-note-1)

[**#techbywebcoder**](#)



Thank you !
Follow For More

